Progetto di High Performance Computing 2024/2025

Gianpiero Tovo, matr. XXXX
12/09/2025

Introduzione

Questo documento presenta il lavoro svolto per lo sviluppo di due implementazioni parallele dell'algoritmo Skyline e fornisce un'analisi dei risultati ottenuti.

Ho realizzato una versione che sfrutta il parallelismo a memoria condivisa con OpenMP ed una seconda versione che impiega il parallelismo su hardware grafico sfruttando CUDA.

Metodologia

I programmi sono stati compilati ed eseguiti su un server che monta un processore Intel Core i7-5820K 3.30GHz dotato di 6 core fisici con *hyperthreading*, 64 GB di memoria RAM e due schede GPU Nvidia GeForce Titan X 1.6GHz 12GB RAM.

Per la raccolta dei dati sperimentali, ciascuna configurazione con la quale è stato lanciato il programma è stata eseguita tre volte e dei tempi ricavati è stata registrata la media matematica.

Il risultato di ogni iterazione è stato validato rispetto ai risultati *ground truth,* generati con la soluzione seriale fornita dalla consegna del progetto.

Il processo descritto è stato automatizzato usando script Bash, mentre i grafici sono stati generati con codice Python usando la libreria `mathplotlib`.

Versione OpenMP

Il dominio del problema comprende un vettore di N punti, memorizzato in C++ come array monodimensionale di interi, che presenta un'alta località spaziale.

La struttura dati suggerisce l'implementazione di un pattern di parallelismo di tipo *partition*, che prevede l'assegnazione a ciascun esecutore di una porzione di dati, la quale elaborazione è indipendente. Ciascun punto può essere infatti comparato con gli altri in modo autonomo dal resto dei dati.

L'accesso all'array contenente la soluzione dello skyline per marcare il punto come dominato/non dominato presenta invece una *race condition* che va gestita in modo adeguato, proteggendo le letture e scritture dei thread.

Descrizione della soluzione

La soluzione parallela è basata su due cicli for principali, racchiusi in una direttiva *omp parallel for*.

Il ciclo principale della soluzione seriale rimane sostanzialmente invariato, ma la lettura del valore *s[j]* (il valore con il quale viene comparato il punto in esame) nel ciclo interno viene protetta con una direttiva *omp atomic read* per evitare conflitti con l'eventuale successiva scrittura, nel caso il punto vada escluso dallo skyline.

La conta dei punti appartenenti alla soluzione viene effettuata con un'operazione di *reduction*, in un secondo ciclo dedicato.

La complessità richiesta per verificare l'appartenenza dei punti allo skyline varia in modo non deterministico. Per far fronte ad un bilanciamento non omogeneo del carico, le iterazioni del primo ciclo esterno vengono allocate in modo dinamico e partizionate a grana fine, utilizzando il *chunksize* di default (`chunksize=1`).

La complessità di un'iterazione del conteggio degli elementi della soluzione è invece costante, ed il ciclo viene quindi parallelizzato con scheduling statico.

Discussione delle prestazioni

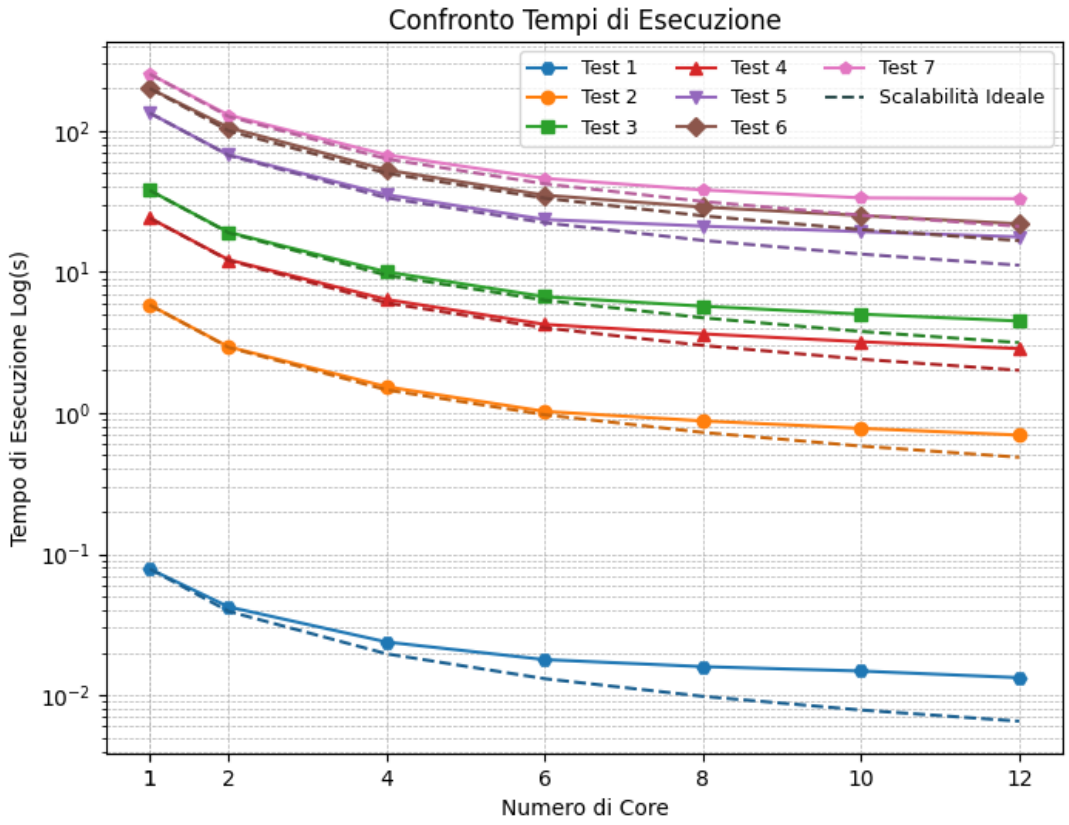
La soluzione sviluppata presenta una buona scalabilità, sia al crescere del carico di lavoro che al crescere del numero di esecutori a disposizione.

L'efficienza tende all'ideale quando vengono utilizzati i core fisici, mentre il miglioramento è decisamente rallentato quando entrano in gioco i core virtuali (numero di core maggiore di sei).

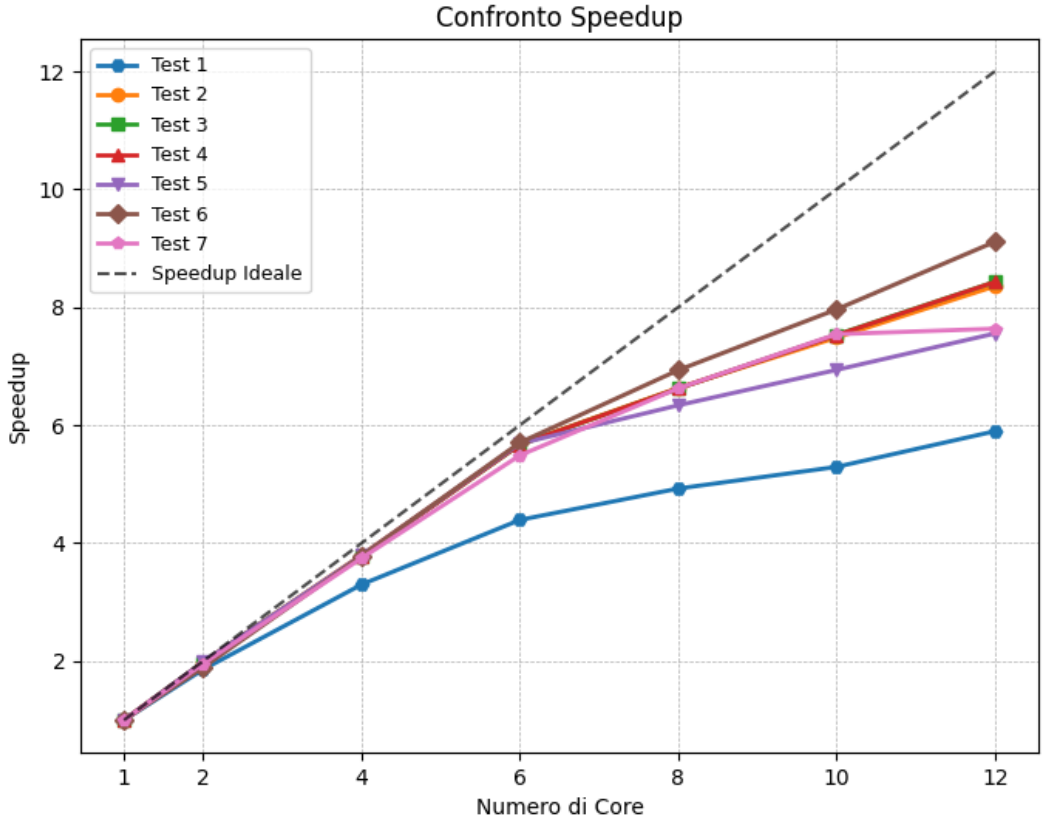
Quest'ultimo aspetto è evidenziato dalla metrica strong scaling efficiency, che indica come l'utilizzo dell'*hyperthreading* introduca un overhead non sostenibile.

Il set `Test1` sembra non avere una dimensionalità tale da beneficiare del parallelismo come gli altri set e presenta i peggiori risultati su tutte le metriche.

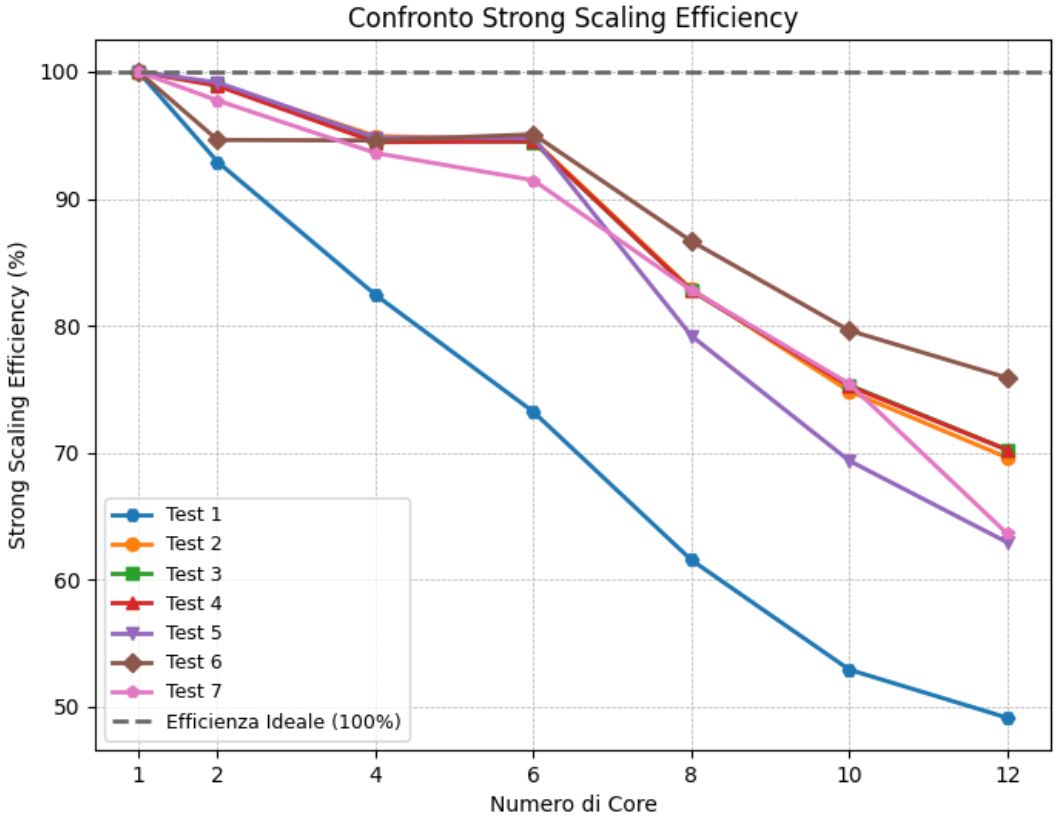
Il set `Test7`, il più complesso per dimensionalità e grandezza della soluzione, riporta un buon miglioramento ma dimostra un rallentamento più marcato rispetto ai test intermedi nell'utilizzo del massimo numero di core.



Il tempo di esecuzione migliora con tutti i set ma il miglioramento rallenta più velocemente con `Test1` e `Test7`.



Il carattere sublineare dello speedup risulta più marcato con `Test1` e `Test7`.



Con le prime configurazioni di core l'efficienza è soddisfacente, ma con l'utilizzo dei core virtuali l'overhead diventa considerevole. Il set `Test1` sembra beneficiare meno del parallelismo fin dall'inizio rispetto agli altri set.

Versione CUDA

La soluzione CUDA opera in modo simile alla versione OpenMP. In particolare, nel kernel *skyline* ciascun *thread* calcola il proprio indice e procede verificando l'appartenenza del relativo punto allo skyline.

L'inizializzazione dell'array delle soluzioni viene effettuata da un kernel richiamato nella funzione *main,* mentre il conteggio degli elementi appartenenti allo skyline viene eseguito serialmente.

Considerato l'overhead introdotto dall'esecuzione di codice non parallelo su hardware grafico, l'analisi è stata condotta rispetto alle prestazioni della soluzione seriale (trattata nella precedente sezione riguardante OpenMP) eseguita su CPU.

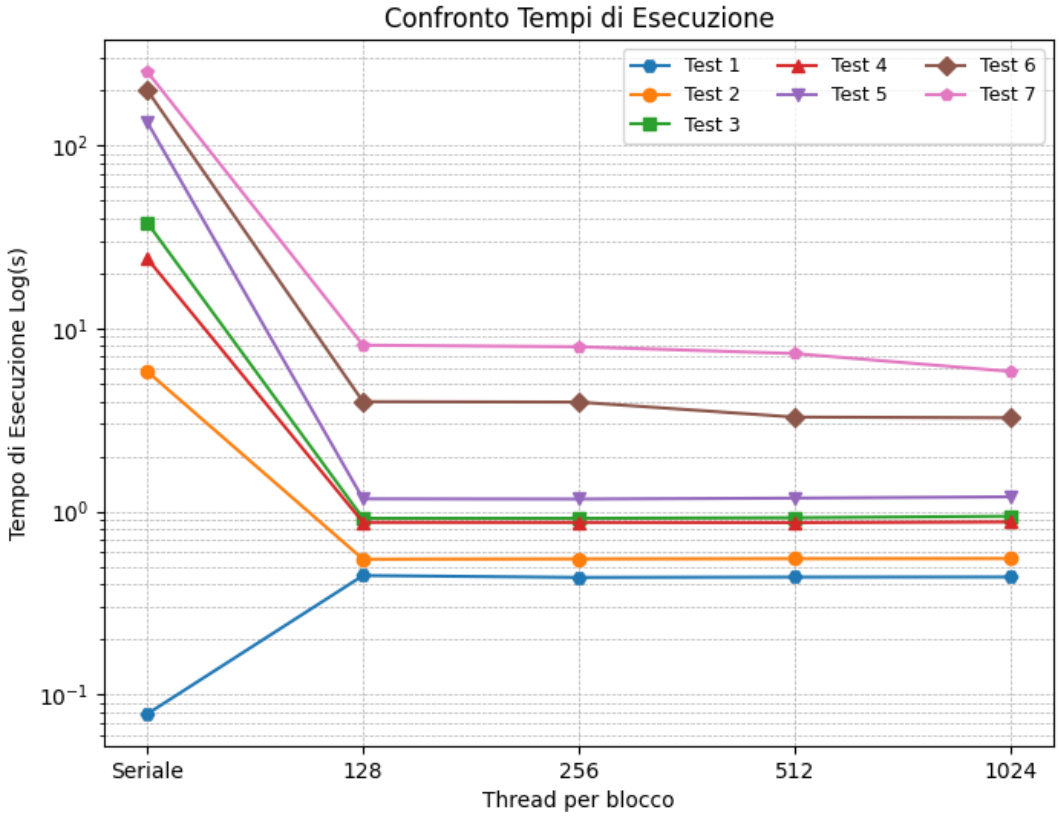
Discussione delle prestazioni

I tempi di esecuzione sono stati registrati eseguendo il programma con tutti i dataset, all'incremento dei thread per blocco.

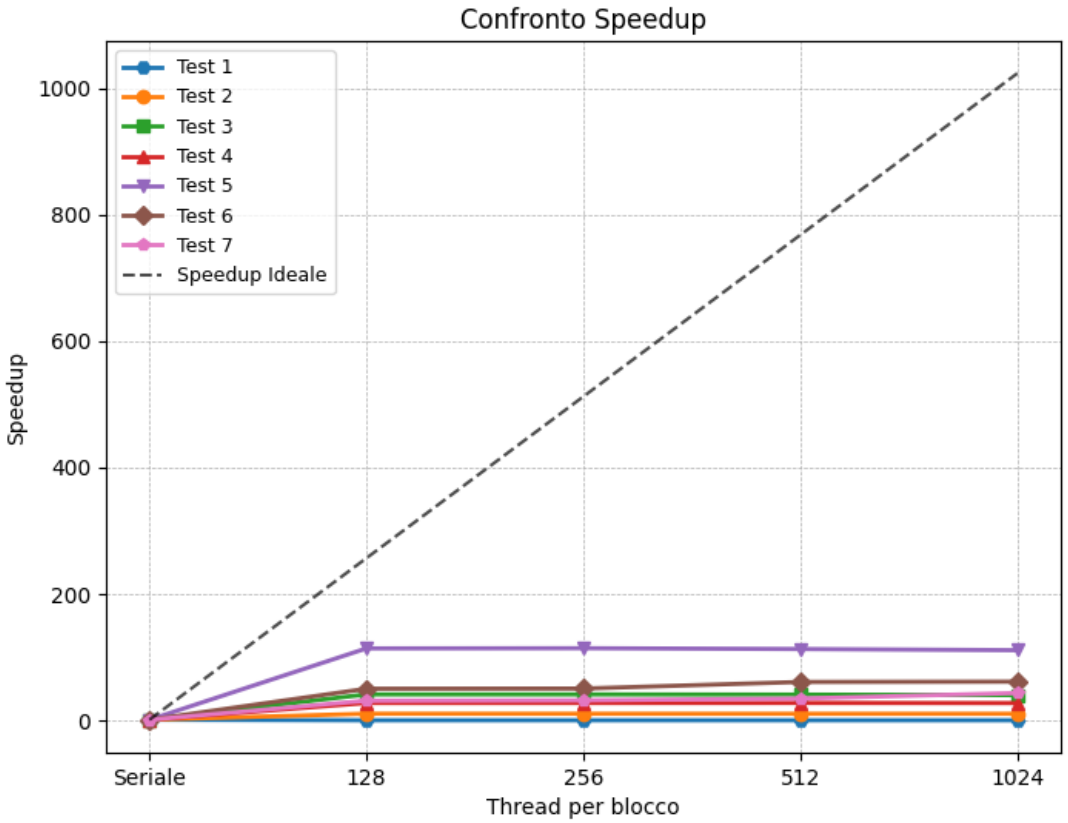
Rispetto al parallelismo con OpenMP su CPU con il quale si nota un miglioramento graduale delle prestazioni, nella soluzione CUDA è evidente la distribuzione uniforme del lavoro tra gli *streaming multiprocessor* e il livello elevato di *occupancy*.

Le prestazioni risultano infatti simili con tutte le configurazioni sperimentate: l'hardware è in grado di gestire facilmente la quantità di lavoro assegnata e gli unici colli di bottiglia risultano essere i trasferimenti e le sezioni di codice seriali.

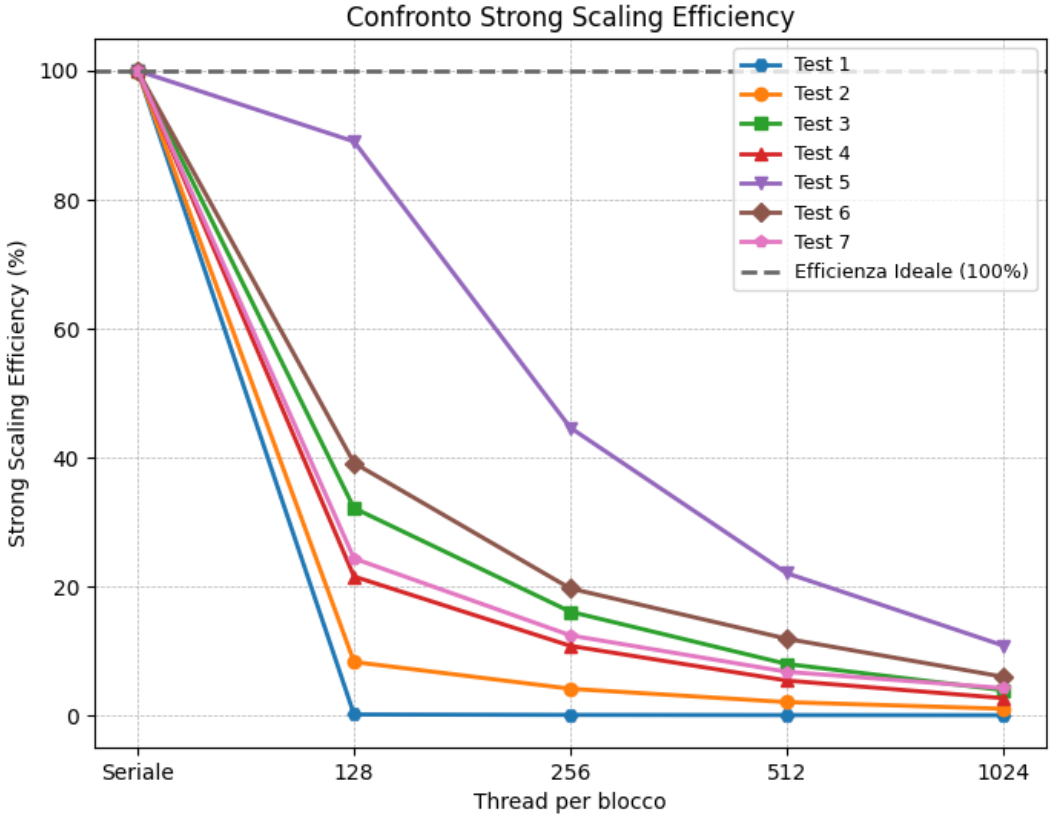
Come visto in precedenza, il set `Test1` sembra non beneficiare del parallelismo e quando risolto su GPU impiega dei tempi di esecuzione addirittura maggiori della versione seriale.



La GPU riesce a gestire in modo efficace la complessità dei dataset e distribuisce agilmente il lavoro tra i *single processors*.



Considerate le prestazioni simili tra le diverse configurazioni, non c'è miglioramento al variare del numero di thread per blocco.



All'aumento di thread per blocco incrementa l'overhead e le prestazioni vengono limitate.

Conclusioni

I due programmi hanno implementato due diversi tipi di parallelismo e hanno dimostrato i comportamenti tipici dei due paradigmi.

La versione OpenMP scala all'aumento di processori disponibili, ma le prestazioni vengono progressivamente limitate dall'overhead, in particolare dall'uso dei core virtuali.

La versione CUDA sfrutta le capacità di parallelismo dell'hardware grafico, mantenendo un alto livello di *occupancy* dei processori e riuscendo a gestire facilmente la complessità del dataset sperimentato.